EXPERIMENT NO 06

20-10-2026

NAME: YUGITHA B

REG NO: 192424102

SLOT: A

SUBJECT CODE: CSA4701

SUBJECT: DEEP LEARNING

SEMESTER: MS2-SEPTEMBER-2026

Gradient Descent, SGD, Momentum and Adam Comparison

SCENARIO

A linear regression model predicts house prices but initially produces high prediction error. Implement gradient descent and visualize how the loss changes over successive iterations.

AIM

To implement gradient descent for a linear regression model predicting house prices and visualize the reduction in loss over successive iterations.

ALGORITHM

1. Import the required Python libraries.
2. Create a sample dataset containing house sizes and prices.
3. Initialize the model parameters and learning rate.
4. Calculate predicted prices and mean squared error.
5. Update the parameters using gradient descent over multiple iterations.
6. Display the final loss and plot the loss reduction across iterations.

SOURCECODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# Sample house dataset
X = np.array([500, 800, 1000, 1200, 1500, 1800, 2000], dtype=float)
y = np.array([15, 22, 28, 32, 40, 48, 52], dtype=float)
```

```
# Normalize house sizes and prices
X = (X - X.mean()) / X.std()
y = (y - y.mean()) / y.std()

# Initialize parameters
w = 0.0
b = 0.0
learning_rate = 0.05
iterations = 500
loss_history = []

# Gradient descent
for i in range(iterations):
    y_pred = w * X + b
    error = y_pred - y

    loss = np.mean(error ** 2)
    loss_history.append(loss)

    dw = 2 * np.mean(error * X)
    db = 2 * np.mean(error)

    w -= learning_rate * dw
    b -= learning_rate * db

# Final predictions and loss
final_predictions = w * X + b
final_loss = np.mean((final_predictions - y) ** 2)

# Summary table
```

```
summary = pd.DataFrame({
   'Measure': ['Initial Loss', 'Final Loss',
               'Final Weight', 'Final Bias'],
   'Value': [loss_history[0], final_loss, w, b]
})

print("GRADIENT DESCENT SUMMARY")
display(summary.round(4))

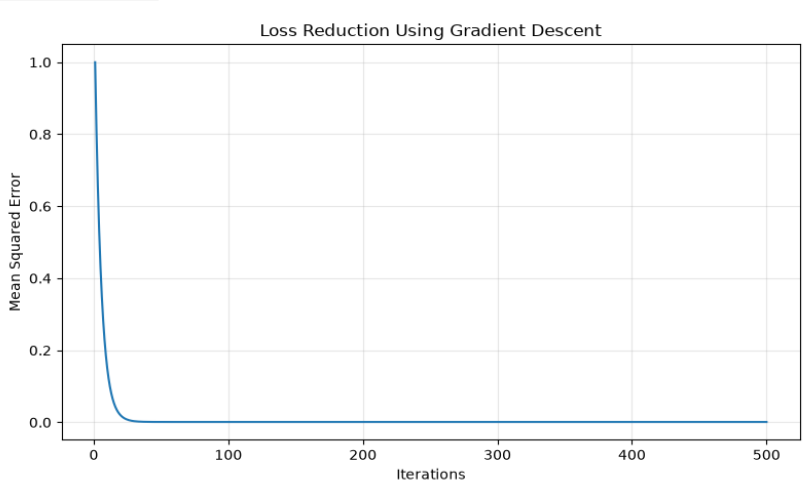
# Plot loss reduction
plt.figure(figsize=(8, 5))
plt.plot(range(1, iterations + 1), loss_history)
plt.xlabel('Iterations')
plt.ylabel('Mean Squared Error')
plt.title('Loss Reduction Using Gradient Descent')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

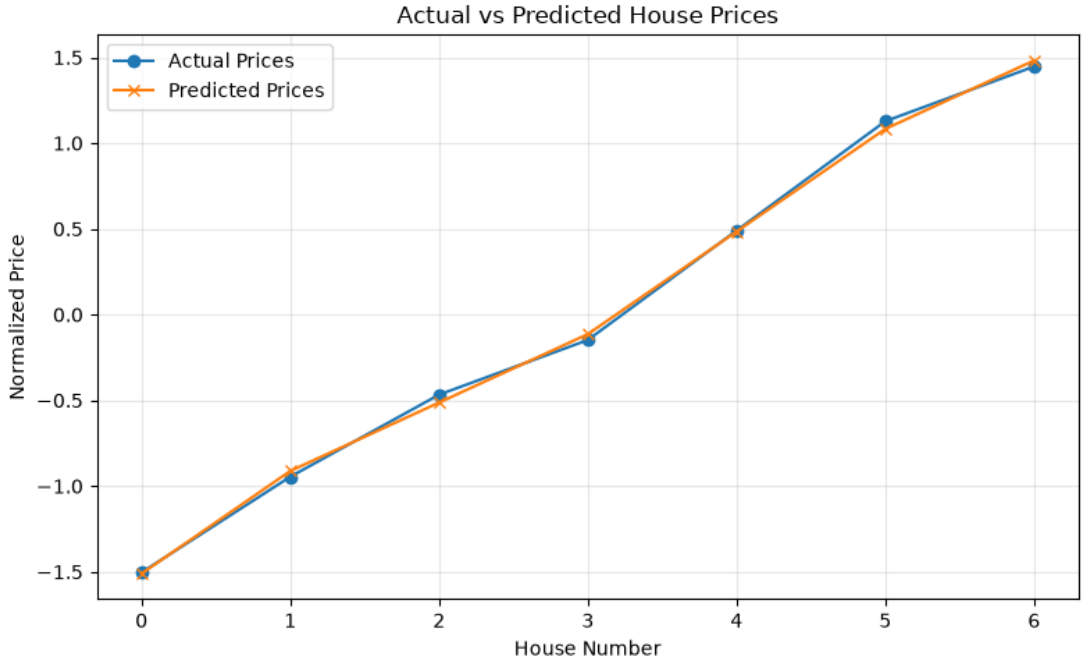
# Compare actual and predicted normalized prices
plt.figure(figsize=(8, 5))
plt.plot(y, marker='o', label='Actual Prices')
plt.plot(final_predictions, marker='x', label='Predicted Prices')
plt.xlabel('House Number')
plt.ylabel('Normalized Price')
plt.title('Actual vs Predicted House Prices')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

OUTPUT

GRADIENT DESCENT SUMMARY

	Measure	Value
0	Initial Loss	1.0000
1	Final Loss	0.0011
2	Final Weight	0.9994
3	Final Bias	0.0000





RESULT

Thus, gradient descent was successfully implemented for linear regression. The loss curve demonstrates how iterative parameter updates reduce prediction error and improve the model's fit to the sample house-price data.